

AudioNEXT

Lautstärkeanalyse und VGA-Visualisierung auf dem Arty A7

Felix Knoll Maximilian Hafeneder Nicolas Lonthoff Markus Remy Richard
Tuch

Team 05 – Die Audiophilen

CSA - Computer System Architektur

25. Juni 2026

Gliederung

Einleitung / Motivation

Notwendige Grundlagen

Aufgabenstellung / Zielsetzung

Lösungsansatz / Konzept

Lösung / Implementierung

Evaluierung / Ergebnisse

Diskussion

Schlusszusammenfassung

Literaturverzeichnis

Quellcodeübersicht

Aufgetretene Probleme

Einleitung / Motivation [Knoll]

- AudioNEXT misst die Lautstärke eines eingehenden Audiosignals in Echtzeit
- Eingang über eine 3,5 mm Klinke und den Pmod I2S2
- Ausgabe der Lautstärke über den Pmod VGA als Text und Balken
- Reiz, ein komplettes System von der Audioquelle bis zur Anzeige selbst aufzubauen
- Eigene Hardware und Software entwerfen und ihr Zusammenspiel verstehen
- Praxisnahe Anwendung der Inhalte aus Computer System Architektur

Notwendige Grundlagen - I²S Protokoll [1, 2] [Hafeneder]

- **I²S**: Serielles digitales Audioprotokoll speziell zur Übertragung von Audiodaten zwischen integrierten Schaltkreisen wie etwa einem FPGA und dem CS5343 (A/D Wandler auf dem Pmod I2S2)
- verwendet drei Signalleitungen:
 - **MCLK** (Master Clock): Haupttakt
 - **LRCK** (Left-Right Clock / Word Select): Kanalauswahl ('0' = links, '1' = rechts)
 - **SCLK** (Serial Clock / Bit Clock): Takt zur seriellen Datenübertragung
- 32 Bit pro Kanal, MSB zuerst
- I²S läuft im Master-Slave Prinzip → FPGA generiert entweder alle Taktsignale oder empfängt sie

Notwendige Grundlagen - I²S Timing [1] [Hafeneder]

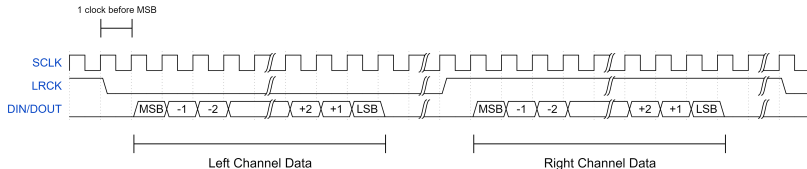


Abbildung 1: I²S Timing-Diagramm [1]

Besonderheit für die Implementierung

Erstes Datenbit (MSB) liegt erst nach einem vollständigen SCLK-Zyklus nach der LRCK-Flanke an → erster SCLK-Zyklus nach LRCK-Flanke wird übersprungen

Notwendige Grundlagen - RMS [Knoll]

- RMS (Root Mean Square) ist der quadratische Mittelwert eines Signals
- Er beschreibt die mittlere Leistung und damit die wahrgenommene Lautstärke
- Werte quadrieren, über einen Block mitteln und die Wurzel ziehen
- Linker und rechter Kanal werden gemeinsam ausgewertet

$$x_{\text{RMS}} = \sqrt{\frac{1}{2N} \sum_{i=1}^N (l_i^2 + r_i^2)}$$

- Ergebnis als Prozent der Vollaussteuerung im 7.2 Festkommaformat
- Ganzzahlige Wurzel (isqrt) für die Berechnung ohne Gleitkomma

Notwendige Grundlagen - VGA Bildaufbau [3, 4] [Remy]

- Auflösung: 640x480@60 Hz
- Signale:
 - HSync
 - VSync
 - Red
 - Green
 - Blue
- Bildaufbau (hor./vert.):
 - Aktiver Bildbereich
 - Frontporch
 - Sync
 - Backporch

Aufgabenstellung / Zielsetzung [Tuch]

Aufgabe: Echtzeit-Lautstärkemessung eines Audiosignals auf dem Arty A7-100T FPGA mit visueller Ausgabe über VGA.

- Audiodaten über 3,5 mm Klinke und Pmod I2S2 empfangen (I2S, 96 kHz/Kanal)
- Zwei eigene AXI Lite Peripherals für Audio und Visualisierung entwerfen
- RMS-Lautstärke auf dem MicroBlaze in Software berechnen
- Ergebnis als Text und Balken auf einem VGA-Monitor darstellen

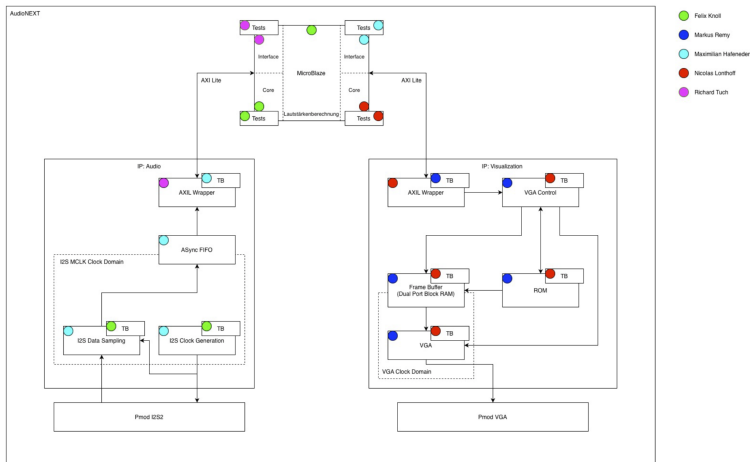
Abgrenzung:

- Keine Gleitkomma-Arithmetik, ausschließlich Festkomma (7.2-Format)
- Datenübertragung per Register-Polling (kein Interrupt-Betrieb im Regelbetrieb)

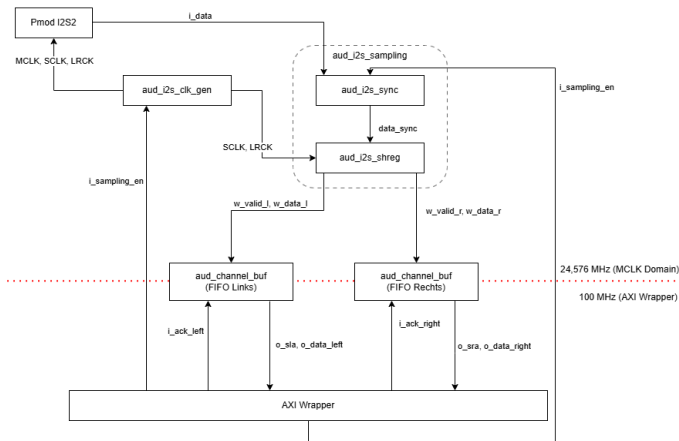
Lösungsansatz / Konzept - [Knoll]

- Aufteilung in zwei eigene AXI IPs und den MicroBlaze als zentrale Steuerung
- Audio IP empfängt das I2S Signal, puffert die Samples und stellt sie bereit
- MicroBlaze berechnet die Lautstärke als quadratischen Mittelwert (RMS)
- Visualization IP erzeugt das VGA Bild aus einem Textframebuffer
- Zeitkritische Verarbeitung in Hardware, flexible Berechnung in Software
- Kommunikation einheitlich über AXI Lite mit Interrupts je Kanal
- Feste Registerschnittstellen ermöglichen paralleles Arbeiten im Team

Lösungsansatz / Konzept - Blockdiagramm [Alle]



Lösung / Implementierung - Audio - Aufbau [Hafeneder]



Lösung / Implementierung - Audio - Takterzeugung [Hafeneder]

- Pmod I2S2 im Slave Mode: MCLK, SCLK und LRCK werden vom FPGA erzeugt
- 24 Bit Nutzungsdaten pro Kanal, Rest '0'
- **MCLK** = 24,576 MHz
- MCLK/LRCK Ratio = 256 → **LRCK** = 96 kHz Samplingrate (Double-Speed Mode)
- SCLK/LRCK Ratio = 64 → **SCLK** = 6,144 MHz
- Implementierung in `aud_i2s_clk_gen.vhd`:
 - SCLK: 1-Bit-Zähler togglet alle 2 MCLK-Takte → Periode = 4 MCLK
 - LRCK: 7-Bit-Zähler togglet alle 128 MCLK-Takte → Periode = 256 MCLK
- Takte nur aktiv wenn `i_sen` = '1' (Sampling ist aktiviert)

Lösung / Implementierung - Audio - Einlesen in Shiftregister [Hafeneder]

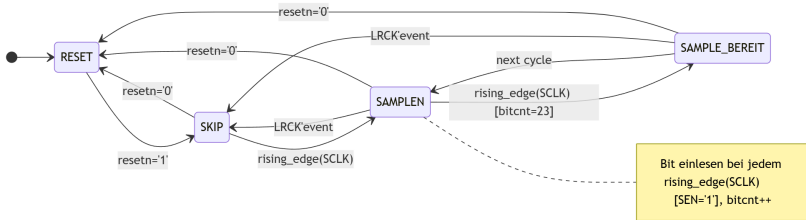


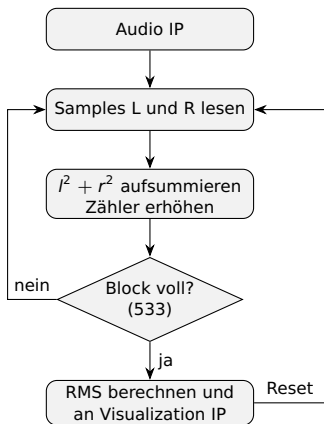
Abbildung 2: Ablauf der Befüllung des I²S-Schieberegisters (aud_i2s_shreg.vhd)

Lösung / Implementierung - Audio - CDC & Pufferung [Hafeneder]

- Zwei Taktdomänen: **24,576 MHz** (Audio (MCLK)) und **100 MHz** (AXI Wrapper)
- `aud_i2s_sync`: 2-Flip-Flop Synchronisierer für das externe, asynchrone Signal `i_data` → verhindert metastabile Zustände
- `aud_channel_buf`: AXI-Stream FIFO (Xilinx FIFO Generator 13.2)
 - Schreibseite: 24,576 MHz (Audio)
 - Leseseite: 100 MHz (AXI)
 - getrennte FIFOs für linken und rechten Kanal
- Data Overrun Flag (DOL/DOR) wenn ein neuer Sample anliegt aber der FIFO voll (nicht bereit) ist

Lösung / Implementierung - Audio Software [Knoll]

- Sampling über das CTRL Register aktivieren
- Statusbits SLA und SRA pollen
- 24 Bit Samples mit Vorzeichenerweiterung lesen
- Pufferüberläufe über DOL und DOR erkennen
- RMS berechnen und an die Visualization IP geben
- Funktionstest und Selbsttests prüfen
Register, Sampling und RMS
Berechnung automatisch beim
Programmstart



Lösung / Implementierung - Audio AXI Hardware [Tuch]

- AXI Lite Slave mit 10 Registern nach dem 5-Teile-Muster aus den Tutorials
- IPISR nutzt TOW (Toggle On Write): Schreiben einer 1 kippt das Statusbit per XOR zurück
- Interrupt-Ausgabe pro Kanal: $GIE \wedge IPIER \wedge IPISR$
- Lesen von ADATLR/ADATRR erzeugt einen Acknowledge-Puls, der den AXI-Stream Handshake mit dem CDC-FIFO auslöst
- Dabei wird `sla_flag` für einen Takt maskiert, damit das IPISR-Bit nicht sofort erneut gesetzt wird
- Selbsttest prüft Register, Sampling Enable und beide Kanäle beim Programmstart

Offset	Register	Zugriff
0x00	GSCR	Reserved
0x04	GIER (GIE)	R/W
0x08	IPIER (SLIE, SRIE)	R/W
0x0C	IPISR (SLA, SRA)	R/W (TOW)
0x10	IDR	R (Konstante)

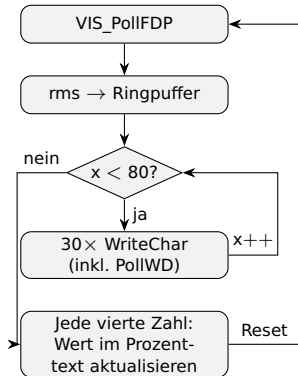
Lösung / Implementierung - Visualization AXI Hardware [Lonthoff]

- AXI Lite Slave mit anderen Bits und einem Register (COLR) mehr als Audio AXI
- CTRL.WD wird von vis_core automatisch gelöscht, sobald der Framebuffer die Übernahme bestätigt (buf_ready = '1')
- Zeichenschreiben: ADDR (X/Y) → VDATR (ASCII) → COLR (RGB) → CTRL.WD=1
- STATUS.FDP = 0 zeigt Ende des sichtbaren Frames → nächster Frame kann übertragen werden
- IPISR.FDP mit W1C Toggle: Schreibe 1 zum 0 setzen

Offset	Register
0x00	GCSR (Reserved)
0x04	GIER (GIE)
0x08	IPIER (FIE)
0x0C	IPISR (FDP)
0x10	IDR (Konstante)
0x14	VERR (Konstante)
0x18	CTRL (VEN, WD)
0x1C	STATUS (FDP)
0x20	ADDR (XA, YA)
0x24	VDATR (CHAR)
0x28	COLR (CR, CG, CB)

Lösung / Implementierung - Visualization Core Software [Lonthoff]

- Ringpuffer mit 80 Werten: ältester Wert links (Spalte 0), neuester rechts (Spalte 79)
- VIS_PollFDP wartet auf STATUS.FDP vor Schreiben des nächsten Frames
- VIS_WriteChar schreibt ein farbiges (COLR) Zeichen (VDATR) an die aktuelle Position (ADDRR) und setzt CTRL.WD=1, dann PollWD bis zurückgesetzt
- Balkenfarbe nach Zeilenposition: grün (unten $\frac{1}{3}$), gelb (Mitte), rot (oben)
- Prozentanzeige: gleitender Mittelwert über die letzten vier 7.2-Festkommazahlen



Lösung / Implementierung - Visualization Core

Software Tests [Lonthoff]

Unit-Tests (kein HW)

- VIS_Core_TestFormat: 7 Grenzwerte der 7.2-Festkomma-Formatierung (0,00 % ... 127,75 %)
- VIS_Core_TestBarWidth: Höhe bei 0 %, 25 %, 50 %, 100 % und Over-Range (Grenze bei Höhe 80)

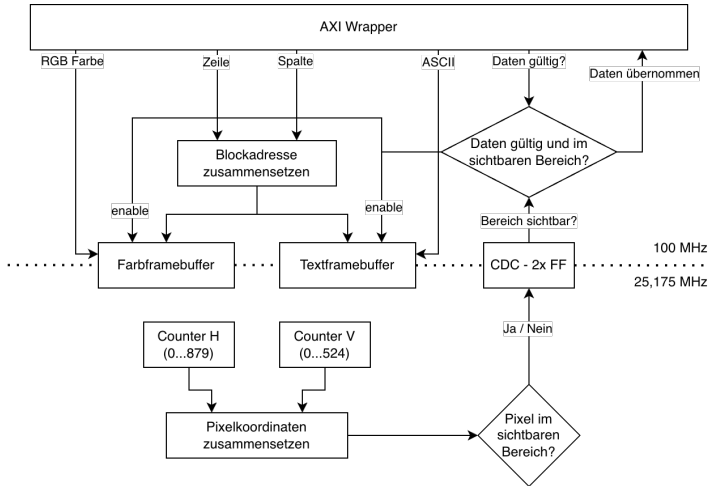
Integrationstests (HW)

- VIS_TestRegisters: alle 11 Register mit 0xFF/0x00; prüft Bitmasken, W1C und Konstanten (IDR, VERR)
- VIS_Core_TestFDP: STATUS.FDP erscheint ≤ 3 Frames; Timeout verhindert Deadlock
- VIS_Core_TestRender: 7 Spalten mit steigendem RMS (0 % ... >100 %); visuelle Prüfung
- VIS_Core_TestScrolling: Rampe prüft korrektes Ringpuffer-Wrap nach 80 Werten; visuelle Prüfung

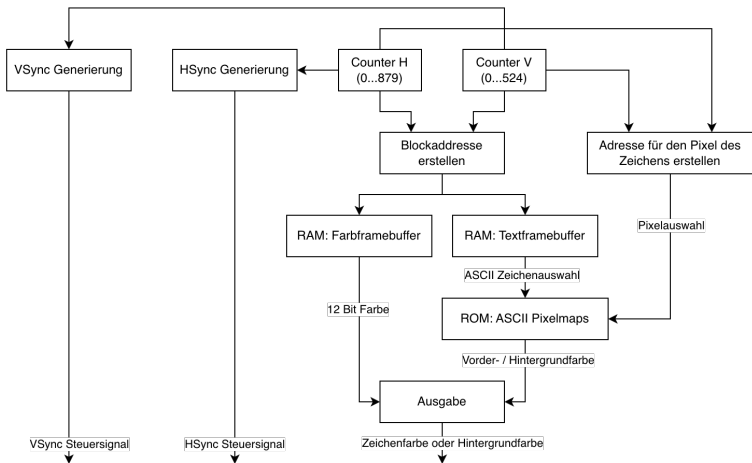
Lösung / Implementierung - VGA - Funktionsübersicht [Remy]

- Bildaufteilung in 8x16 Pixel Blöcke für je ein Zeichen
- Schreiben neuer Werte in den Farb- & Textframebuffer während des nicht sichtbaren Bereichs
- Textframebuffer: Verweis auf ASCII-Zeichen im ROM pro Block
- Farbframebuffer: 12 Bit Textfarbe pro Zeichenblock
- ROM: 8x16 Pixelmap pro Zeichen (1 = Text, 0 = Hintergrund)
- Pmod VGA zur Ausgabe des Signals

Lösung / Implementierung - VGA - Steuerung [Remy]



Lösung / Implementierung - VGA - Bildausgabe [Remy]



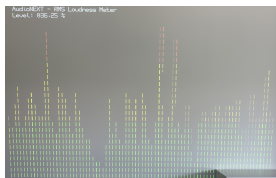
Evaluierung / Ergebnisse [Lonthoff]

- System läuft stabil auf dem Arty A7-100T
- VGA-Ausgabe auf 3 verschiedenen Displays verifiziert (Beamer K101, Beamer K143, Monitor K143a)
- SW-Selftests automatisch beim Start: 11 Register, Formatierung (7 Grenzwerte), Balkenhöhe (6 Fälle), Rendern (7 Werte), FDP-Protokoll

Ziel (aus Zielsetzung)	Erfüllt	Anmerkung
I2S-Stereo-Sampling via Pmod I2S2 (96 kHz/Kanal)	ja	24-Bit, Async-FIFO
Zwei eigene AXI Lite IPs (Audio + Visualisierung)	ja	10 bzw. 11 Register
RMS-Lautstärke in Software (Festkomma, kein Float)	ja	7.2-Format, 533 Samples/Block
Text- und Balkenanzeige via VGA (640×480@60 Hz)	ja	3 Farbzonen, 80 Spalten durchlaufend
Kein DMA, reines Register-Polling	ja	PollWD/PollFDP/LR_Available

Evaluierung / Ergebnisse - Getestete Bildschirme [Remy]

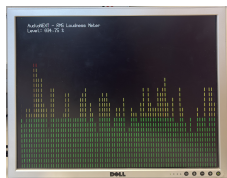
Beamer in K101



Beamer in K143



Bildschirm aus K143a



Diskussion [Tuch]

- **Bedeutung der Ergebnisse:**

- Vollständige Pipeline von der Audioquelle bis zur VGA-Anzeige funktionsfähig
- Eigene AXI IPs und Treiber zeigen das Zusammenspiel von HW und SW auf einem realen FPGA
- Selbsttests sichern die Kernfunktionen automatisch ab

- **Ausblick:**

- Interrupt-gesteuertes Sampling statt Polling für geringere CPU-Last
- Frequenzanalyse (FFT) für ein Spektrogramm auf dem VGA-Ausgang

- **Offene Punkte:**

- Auslastungsanalyse: Wie viel Puffer bleibt beim Schreiben in den CDC-FIFO, bevor Samples verloren gehen?
- Interrupt-Pfad ist in der Hardware vorbereitet (GIER, IPIER, IPISR), wird in Software aber noch nicht genutzt

Schlusszusammenfassung - [Knoll]

- AudioNEXT zeigt die aktuelle Lautstärke live auf einem VGA Monitor
- Audioerfassung, RMS Berechnung und Visualisierung greifen zuverlässig ineinander
- Eigene AXI IPs und passende Softwaretreiber wurden entwickelt
- Selbsttests und Funktionstests sichern die einzelnen Komponenten ab
- Zusammenspiel von Hardware und Software am realen FPGA System gezeigt
- Klare Modulgrenzen und Schnittstellen haben die Integration vereinfacht

Literaturverzeichnis

- [1] Digilent. Pmod I2S2 Reference Manual. URL: <https://digilent.com/reference/pmod/pmodi2s2/reference-manual> abgerufen am 25.06.2026.
- [2] NXP Semiconductors. UM11732 I2S bus specification. User Manual. 17. Feb. 2022. URL: <https://www.nxp.com/docs/en/user-manual/UM11732.pdf> abgerufen am 25.06.2026.
- [3] Digilent. Pmod VGA Reference Manual. URL: <https://digilent.com/reference/pmod/pmodvga/reference-manual> abgerufen am 23.06.2026.
- [4] Digilent. VGA Display Controller. URL: <https://digilent.com/reference/learn/programmable-logic/tutorials/vga-display-controller/start> abgerufen am 21.06.2026.

Literaturverzeichnis

- [5] Digilent. Adding DDR Memory to a Microblaze Design. URL: <https://digilent.com/reference/programmable-logic/guides/microblaze-adding-ddr> abgerufen am 21.06.2026.
- [6] Digilent. digilent-xdc. URL: <https://github.com/Digilent/digilent-xdc> abgerufen am 21.06.2026.
- [7] Daniel Münch und Sandro Amato. VGA Demoprojekt.
- [8] Daniel Münch. Axi Timer Demoprojekt.
- [9] Xilinx. Hello World Demoprojekt.

Quellcodeübersicht - Felix Knoll [Knoll]

- Dateien basierend auf dem Axi Timer Demoprojekt [8]:
 - aud_driver.c, aud_driver.h
 - aud_driver_g.c, aud_driver_i.h
 - aud_driver_l.c, aud_driver_l.h
 - aud_driver_sinit.c, aud_driver_sinit.h
 - aud_selftest_pio.c, aud_selftest_pio.h
- Dateien basierend auf dem Hello World Demoprojekt [9]:
 - audio_rms_main.c
 - platform.c, platform.h
- Dateien ohne Basis:
 - audio_rms.c, audio_rms.h
 - audio_rms_test.c

Quellcodeübersicht - Maximilian Hafeneder 1/2 [Hafeneder]

- Dateien basierend auf dem Axi Timer Demoprojekt [8]:
 - Quellcode/src/FPGA_firmware/aud_tb.sv
 - Quellcode/src/FPGA_firmware/aud_axislave_v1_0.vhd
 - Quellcode/src/Software/vis_selftest_main.c
 - Quellcode/src/Software/vis_selftest.c/.h
 - Quellcode/src/Software/vis_event.c/.h
 - Quellcode/src/Software/vis_driver_sinit.c/.h
 - Quellcode/src/Software/vis_driver_l.c/.h
 - Quellcode/src/Software/vis_driver_g.c
 - Quellcode/src/Software/vis_driver.c/.h
 - (Quellcode/src/Software/vis_driver_int.c/.h)
 - (Quellcode/src/Software/vis_driver_i.h)
 - (aud_db.bd)

Quellcodeübersicht - Maximilian Hafeneder 2/2 [Hafeneder]

- Dateien basierend auf der Digilent Constraints Vorlage [6]:
 - Quellcode/src/FPGA_firmware/ArtyA7100.xdc
- Dateien ohne Basis:
 - Quellcode/src/FPGA_firmware/aud_core.vhd
 - Quellcode/src/FPGA_firmware/aud_i2s_clk_gen.vhd
 - Quellcode/src/FPGA_firmware/aud_i2s_sampling.vhd
 - Quellcode/src/FPGA_firmware/aud_i2s_shreg.vhd
 - Quellcode/src/FPGA_firmware/aud_i2s_sync.vhd
- mit dem FIFO-Generator 13.2 in Xilinx erstellt:
 - Quellcode/src/FPGA_firmware/aud_channel_buf.xci (AXI-Stream FIFO)

Quellcodeübersicht - Nicolas Lonthoff [Lonthoff]

- Dateien basierend auf dem Axi Timer Demoprojekt [8]:
 - vis_axislave_v1_0_S00_AXI.vhd
 - vis_axislave_v1_0.vhd
 - vis_driver.c, vis_driver.h (teils)
 - vis_selftest.c, vis_selftest.h (teils)
- Dateien basierend auf dem Hello World Demoprojekt [9]:
 - audio_rms_main.c (teils)
- Dateien ohne Basis:
 - vis_core.c, vis_core.h

Quellcodeübersicht - Markus Remy [Remy]

- Dateien basierend auf dem VGA Demoprojekt [7]:
 - Quellcode/src/FPGA_firmware/vis_core.vhd
 - Quellcode/src/FPGA_firmware/vis_RAM.vhd
 - Quellcode/src/FPGA_firmware/vis_vga_ctrl.vhd
 - Quellcode/src/FPGA_firmware/vis_vga_pkg.vhd
- Dateien basierend auf dem Axi Timer Demoprojekt [8]:
 - Quellcode/src/FPGA_firmware/vis_tb.vs
 - (vip_db.bd)
 - Quellcode/src/FPGA_firmware/aud_axislave_v1_0.vhd
- Dateien basierend auf der Digilent Constraints Vorlage [6]:
 - Quellcode/src/FPGA_firmware/ArtyA7100.xdc
- Dateien basierend auf der Digilent DDR Vorlage [5]:
 - (mb_blockdesign.bd)

Quellcodeübersicht - Richard Tuch [Tuch]

- Dateien basierend auf dem Axi Timer Demoprojekt:
 - `aud_axislave_v1_0_S00_AXI.vhd`
 - `aud_selftest_pio.c`, `aud_selftest_pio.h`
- Dateien basierend auf dem Hello World Demoprojekt:
 - `audio_rms_main.c` (Beiträge)
- Dateien ohne Basis:
 - `implementation_plan.md`

BACKUP

Aufgetretene Probleme [Remy][Knoll][Lonthoff]

- Constraints:
 - Doppelte Clock Constraints durch automatische von MIG des DDRs generierte und manuell angelegte eigene Constraints
- Cache:
 - Cache in HW aktiviert, aber in SW nicht automatisch aktiviert
 - > DDR Zugriff dauerte sehr lange
 - > Manuelle Aktivierung notwendig
- VGA Timing:
 - Front- und Backporch in der VGA Vorlage vertauscht
 - > Leicht verschobenes Bild
 - Timing in der VGA Vorlage nicht exakt nach den Spezifikationen
 - > Manche Bildschirme interpretieren das Bild falsch
- Balkendarstellung (Lonthoff):
 - Erster Balken erscheint als neuester Wert
 - > Keine zuordbare Fehlerquelle